# Artificial Intelligence

**AIML320/AIML420**

**Preprocessing**

Dr. Heitor Murilo Gomes
School of Engineering and Computer Science
Adaptive AI Lab
heitor.gomes@vuw.ac.nz
http://www.heitorgomes.com

# Outline

- Recap
  - Understanding and transforming features
- Reducing the feature space
  - Feature Selection
- Evaluation (cont.)
  - Metrics (recap)
  - K-fold cross validation
- ~~Imbalanced Classification~~
  - ~~Undersampling, oversampling and cost-sensitive~~

Moved to the next lecture

# Recap

# Understanding and transforming features

- Good learning models require **informative features**
  - if the features do not contain useful information, no learning algorithm can recover it
- Understanding **types** (categorical, numerical, ordinal, etc.) helps determine appropriate <u>preprocessing steps</u> and learners
- Proper preprocessing must avoid **data leakage**
  - compute statistics (e.g. mean, min/max) on the training set and apply the same transformation to the test set

The next question: **do we really need all available features?**

# Reducing the feature space

# Reducing the number of features

- Reducing features can also improve training efficiency, model simplicity, and interpretability
- Too many features (high-dimensionality) can harm performance due to **curse of dimensionality** and increased noise
  - In high-dimensional spaces, points become sparse and distances become less meaningful
  - Harmful for distance-based methods like kNN

**Two main approaches*:**

- <u>Feature selection:</u> keep a subset of the original features
- <u>Dimensionality reduction:</u> transform the feature space into fewer variables (less interpretable)

* In this lecture we focus on feature selection, while dimensionality reduction will be covered later

# Feature selection

- The goal is to identify the most <u>informative</u> features while removing <u>irrelevant</u> or <u>redundant</u> features

**Three main families of approaches:**

- **Filter methods**
  - statistical measures such as correlation or mutual information
- **Wrapper methods**
  - evaluate subsets using a learning algorithm
- **Embedded methods**
  - feature selection occurs during model training

# Feature Selection: **Filter methods**

- Select features using statistical properties of the data, independent of any learning algorithm
- Common criteria include correlation, mutual information, chi-square, or variance
- Very fast and scalable, but ignores interactions between features

# Feature Selection: **Filter methods**

- Select features using statistical properties of the data, independent of any learning algorithm
- Common criteria include correlation, mutual information, chi-square, or variance
- Very fast and scalable, but ignores interactions between features

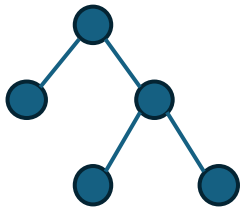
**Step 1:** Calculation correlation between features and target

| Feature | Correlation with target |
|---|---|
| word_freq_your* | 0.383234 |
| word_freq_000 | 0.334787 |
| word_freq_remove | 0.332117 |
| char_freq_%24 | 0.323629 |
| … | … |
| word_freq_will | 0.007741 |

**Step 2:** Keep only the top correlated (filter)

| Feature |
|---|
| word_freq_your |
| word_freq_000 |
| word_freq_remove |

**Step 3:** Train model using them



For each feature, we compute the correlation with the class label.
Features that are more strongly correlated with the target are considered more informative.
* Emails labeled as spam tend to contain the word “your” more often than legitimate emails.

# Feature Selection: **Wrapper methods**

- Evaluate subsets of features using a machine learning model
- The model's performance determines which features are selected
- May be more accurate but computationally expensive

Try all possible subsets of features (showing only subsets of size 3)

| Feature subset | Accuracy |
|---|---|
| {word_freq_address, word_freq_all, word_freq_our} | 0.7487 |
| {word_freq_address, word_freq_3d, word_freq_our} | 0.7408 |
| {word_freq_make, word_freq_all, word_freq_our} | 0.7371 |
| {word_freq_make, word_freq_address, word_freq_our} | 0.7357 |
| {word_freq_address, word_freq_our, word_freq_over} | 0.735 |
| … | … |
| {word_freq_make, word_freq_address, word_freq_3d} | 0.7009 |
| {word_freq_make, word_freq_3d, word_freq_over} | 0.698 |
| {word_freq_address, word_freq_all, word_freq_3d} | 0.698 |
| {word_freq_make, word_freq_all, word_freq_3d} | 0.6959 |

# Feature Selection: **Wrapper methods**

- Issue with this approach?

Number of subsets of size *i*: $\binom{d}{i} = \frac{d!}{i!(d-i)!}$ Total subsets from size 1 to *k*: $\sum_{i=1}^{k} \binom{d}{i}$

All possible non-empty subsets: $$\sum_{i=1}^{d} \binom{d}{i} = 2^d - 1$$

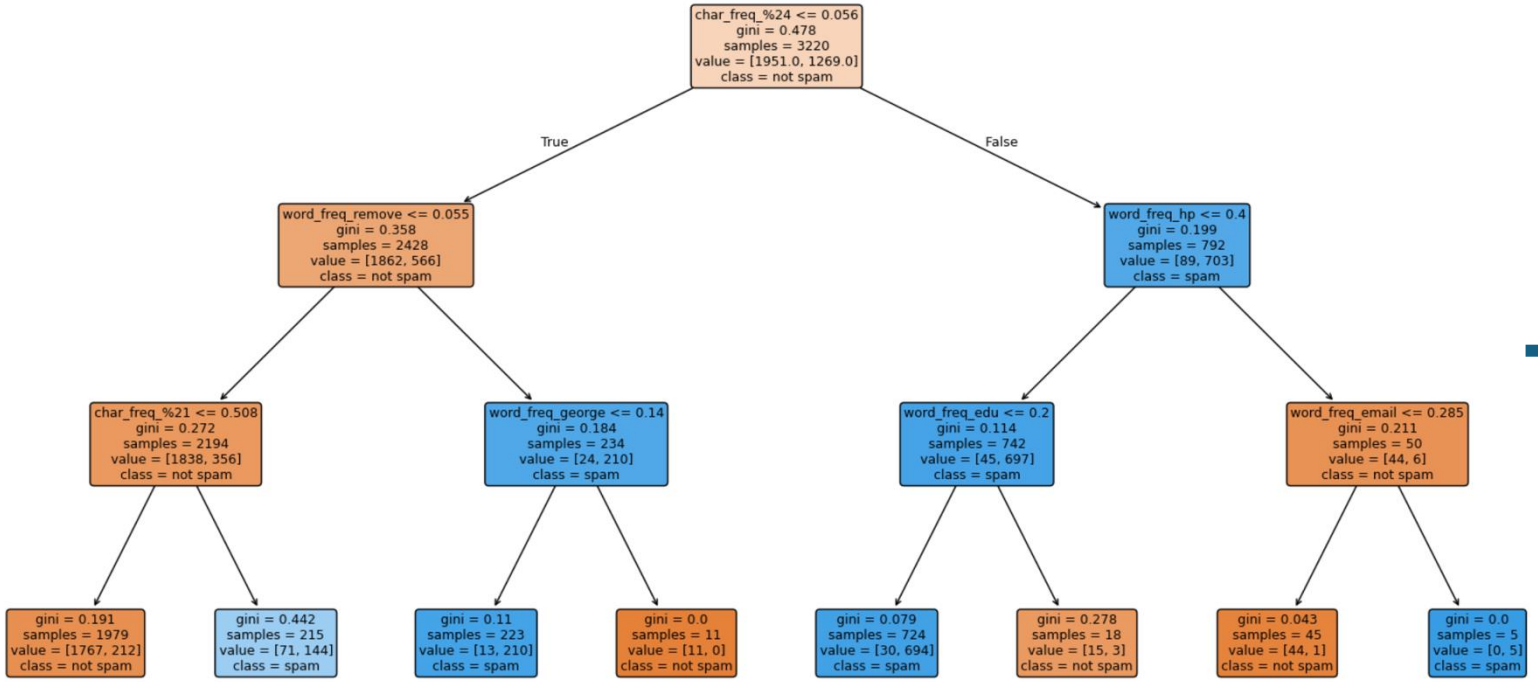
Given 10 features… we need to run the model:

$$\binom{10}{1} + \binom{10}{2} \cdots \binom{10}{10} = 2^{10} - 1 = 2^{10} - 1$$

**1023 times…**

# Feature Selection: **Embedded methods**

- Feature selection occurs during model training
- The learning algorithm automatically identifies important features
- Canonical example: decision trees




**Features used**

char_freq_%21
char_freq_%24
word_freq_edu
word_freq_email
word_freq_george
word_freq_hp
word_freq_remove

# Feature selection remarks

- Reducing the number of features can improve model performance, interpretability, and computational efficiency

- **Filter methods**
  - fast and scalable but may ignore interactions between features
- **Wrapper methods**
  - may find better subsets but are computationally expensive
- **Embedded methods**
  - perform feature selection during model training

- Feature selection is less critical than it once was, especially with modern models that can handle high-dimensional data
- Still useful when interpretability is relevant, in many other cases, **dimensionality reduction** or **learned feature representations** are preferred

# Evaluation (cont.)

# Evaluation

- **Evaluation strategy** and **Evaluation Metric**
  - Examples:
    - Strategy: **holdout (test and train splits)**
    - Metric: **Accuracy**
  - The **strategy** is how we do it, the **metric** is what we measure

- Different metrics for different tasks and problems
  - Choosing the correct evaluation metric is very important
  - The task at hand (classification or regression?)
  - The problem characteristics (much more instances of one class?)
  - What is important for this problem? (depends on the domain)

# Evaluation Metrics

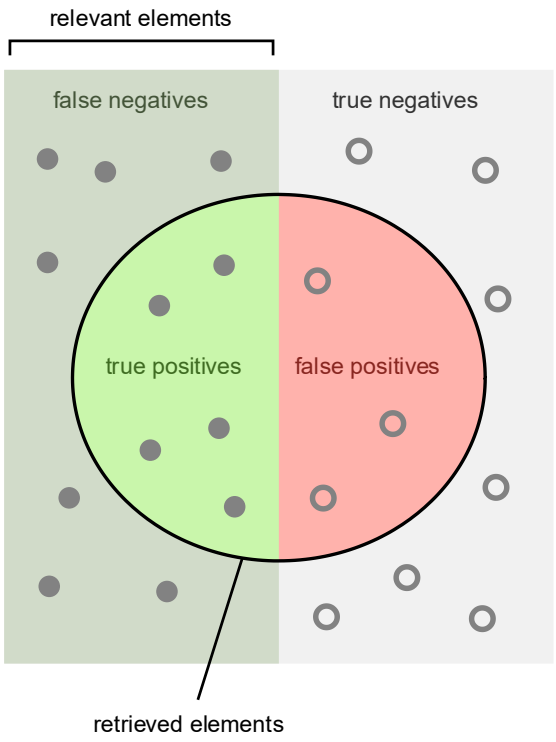
| Predicted Values | Actual Values: Positive (1) | Actual Values: Negative (0) |
|---|---|---|
| Positive (1) | TP | FP |
| Negative (0) | FN | TN |

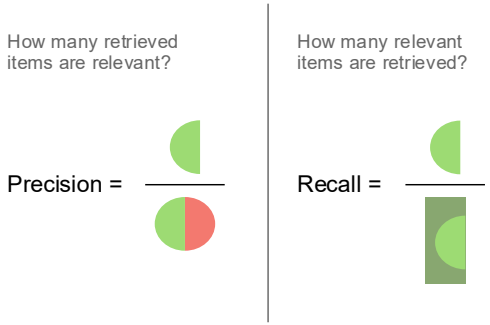
Confusion matrix

TP: True Positive; FP: False Positive;
FN: False Negative; TN: True Negative

**There are several metrics…**

- Accuracy
- Precision
- Recall
- F1-score
- Kappa
- …



# Evaluation Metrics

## Example

- Problem: classify instances as **SPAM (negative)** or **HAM (positive)**; test data contains 100 instances;
- Suppose an algorithm obtains 95% accuracy. Is this good?

Accuracy = (TP+TN) / (TP+TN+FN+FP)

# Evaluation Metrics

## Example

- Problem: classify instances as **SPAM (negative)** or **HAM (positive)**; test data contains 100 instances;
- Suppose an algorithm obtains 95% accuracy. Is this good?

<u>What if 95 of the instances in the test data are **SPAM?**</u>

Accuracy = (TP+TN) / (TP+TN+FN+FP)

# Evaluation Metrics

## Example

- Problem: classify instances as **SPAM (negative)** or **HAM (positive)**; test data contains 100 instances;
- Suppose an algorithm obtains 95% accuracy. Is this good?

<u>What if 95 of the instances in the test data are **SPAM?**</u>

**<u>It depends</u>**

Accuracy = (TP+TN) / (TP+TN+FN+FP)

# Evaluation Metrics

- **Accuracy** does not tell us anything about how accurate the model is in predicting one specific class. In our example, we would like to know if the model can correctly predict **HAM** (positive class)

- Given that 95 of 100 test instances are **SPAM** and the accuracy is 95% it is likely that the model is simply always predicting **SPAM** (negative class)

- In these situations, we need to use more suitable metrics. In our example, we could use **recall***

* recall = TP / (TP+FN)

# Evaluation Metrics

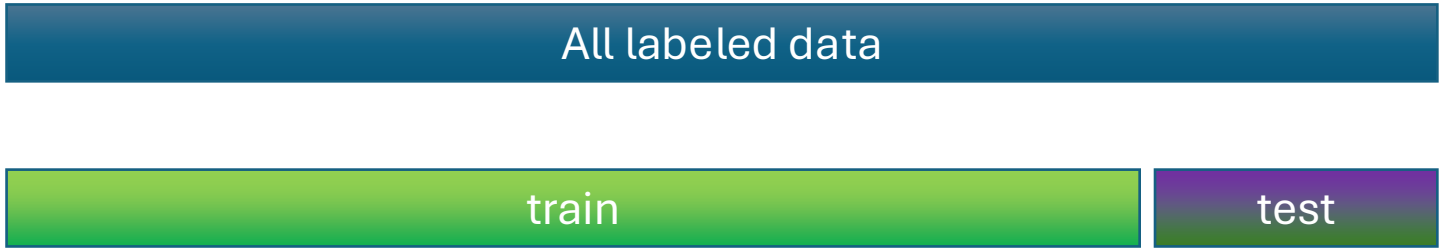
The **choice** of metric can significantly **influence** the **perception of a model's performance**, potentially **overstating** the capabilities of a weak model or **underestimating** the effectiveness of a strong one

**Choose wisely!**

# Evaluation strategy

*Holdout evaluation*

- To address **overfitting**, we should evaluate using data that was not used for training (i.e. test data)




- The aim is to evaluate the model's ability to **generalize** to new and unseen data by testing it with previously unused data.

# Is holdout evaluation enough?

- What are the main **limitations** of the *Holdout Evaluation*?

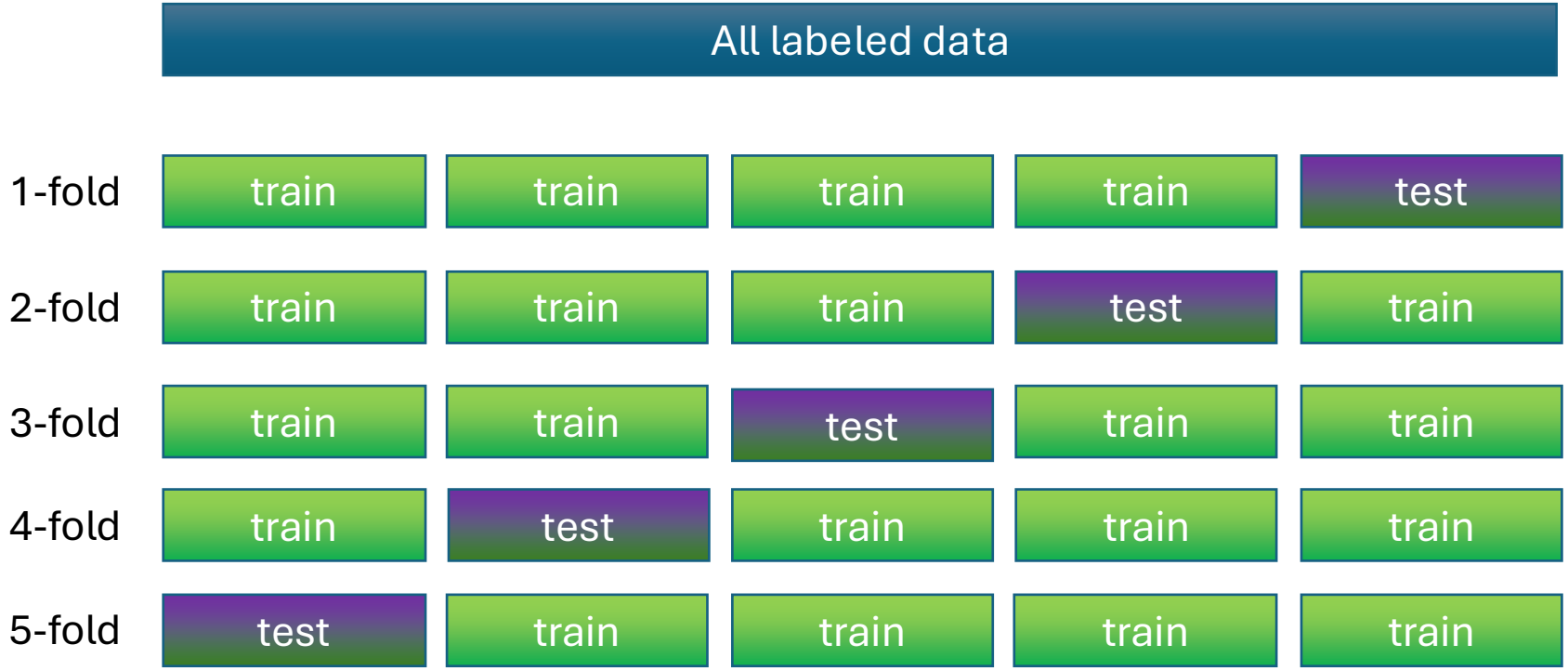
# Is holdout evaluation enough?

- What are the main **limitations** of the *Holdout Evaluation*?
  - Holdout evaluation **may be biased** if the testing dataset is **not representative of the overall population**
  - Having a **small dataset** results in a smaller test set, which can **lower our confidence** in the experimental results

**K-fold cross validation** can help us mitigate these limitations!

# k-fold cross validation

- Create k train and test sets, e.g. 5-fold CV



# k-fold cross validation

- k-fold CV allows us to make **efficient use of the available data** by using <u>each data point for both training and testing</u>, leading to a better estimate of model performance

- **More reliable estimate of performance** especially when the dataset is small or there is a high variance in the data

- **Reduces the risk of overfitting**, as it tests the model on multiple independent test sets, preventing the model from becoming too specific to the training data

# k-fold cross validation

- **<u>Leave-one out:</u>** Extreme case of k-fold CV
  - Create as many test sets as there are data points, each test set contains only one instance
  - It can be useful when the dataset is very small
  - Computationally intensive
  - Can lead to overfitting in some cases

- In overall, we use 5-fold CV or 10-fold CV
  - It depends on the computational resources available
  - More folds leads to more confidence on the results

- Stratified k-fold CV
  - take into account the distribution of the class labels

# Imbalanced classification

# Imbalanced classification

- In many real classification problems, one class is much rarer than the others (e.g. spam detection, medical diagnosis)

- **Accuracy** alone can be misleading in these scenarios (see the evaluation metrics from earlier)

<u>Learning algorithms often favour the majority class</u>

# Imbalanced classification

<u>Learning algorithms often favour the majority class</u>

- If the model is already correctly classifying the majority of examples, the remaining minority cases may contribute little to the optimisation objective

- Consequently, the learner may not prioritise learning additional “rules” to correctly classify those rare instances

# Imbalanced classification

<u>Learning algorithms often favour the majority class</u>

- If the model is already correctly classifying the majority of examples, the remaining minority cases may contribute little to the <u>optimisation objective</u>

- Consequently, the learner may not prioritise learning additional “rules” to correctly classify those rare instances

**Example**

Suppose a decision tree with just one node (the root)

Class 0: 10 examples

Class 1: 100,000 examples

# Imbalanced classification

<u>Learning algorithms often favour the majority class</u>

- If the model is already correctly classifying the majority of examples, the remaining minority cases may contribute little to the <u>optimisation objective</u>
- Consequently, the learner may not prioritise learning additional “rules” to correctly classify those rare instances

**Example**

Suppose a decision tree with just one node (the root)

Class 0: 10 examples

Class 1: 100,000 examples

This node is already **<u>almost pure</u>**

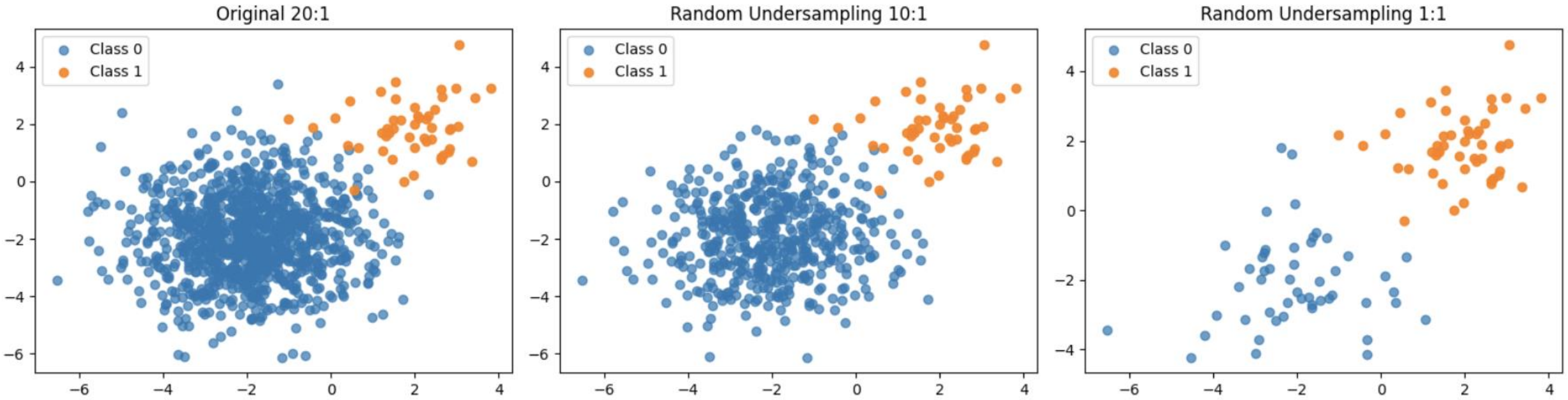
Why split further?
Any split that isolates the minority class will only produce a **very small reduction in impurity**, so depending on the stopping criteria the tree will simply not grow

# Imbalanced classification

- What can we do to address class imbalance?
  - **Undersampling** the majority class
    - Remove instances from the majority
  - **Oversampling** the minority class
    - “Create” instances for the minority
  - **Cost-sensitive learning** (i.e.. class weighting)
    - Modify the <u>optimisation objective</u> to penalise mistakes on the minority class more heavily

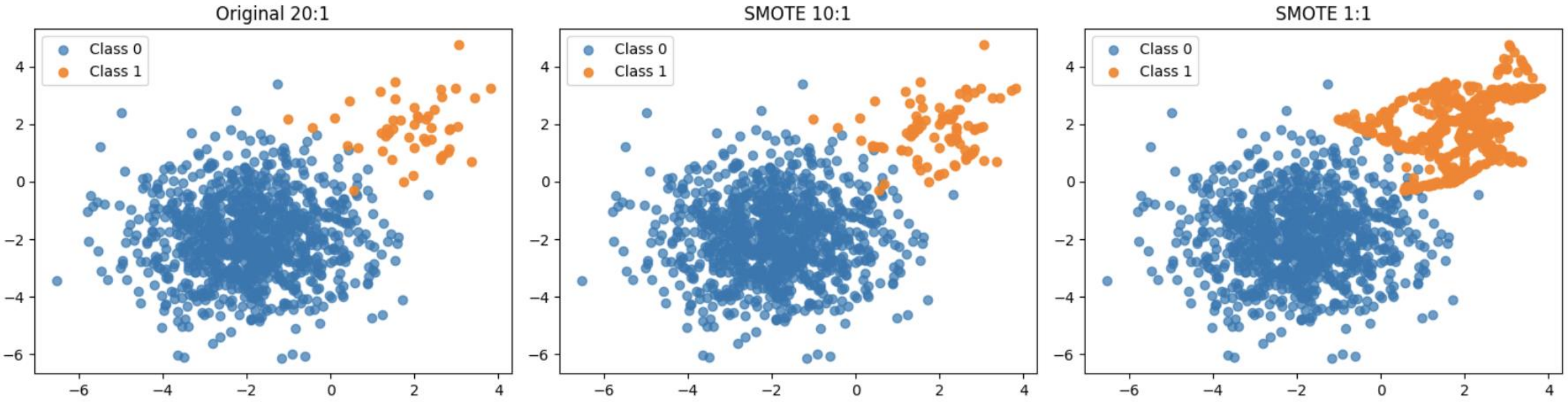
# Undersampling

- Randomly remove instances from the majority class
- Reduces or eliminates class imbalance in the training dataset
- May discard useful information, since many majority examples are removed



# Oversampling

- Add instances to the minority class to reduce class imbalance
- Makes minority cases appear more frequently during training
- Synthetic approaches (e.g. SMOTE[1]) create new examples based on existing minority samples




[1] SMOTE: synthetic minority over-sampling technique NV Chawla, KW Bowyer, LO Hall, WP Kegelmeyer. Journal of artificial intelligence research, 2002

# **Oversampling:** SMOTE

- SMOTE = Synthetic Minority Over-sampling Technique)
- It creates new minority examples by interpolating between nearby minority points in feature space
- It creates new points <u>between existing minority samples</u>, expanding the region of the feature space occupied by the minority class

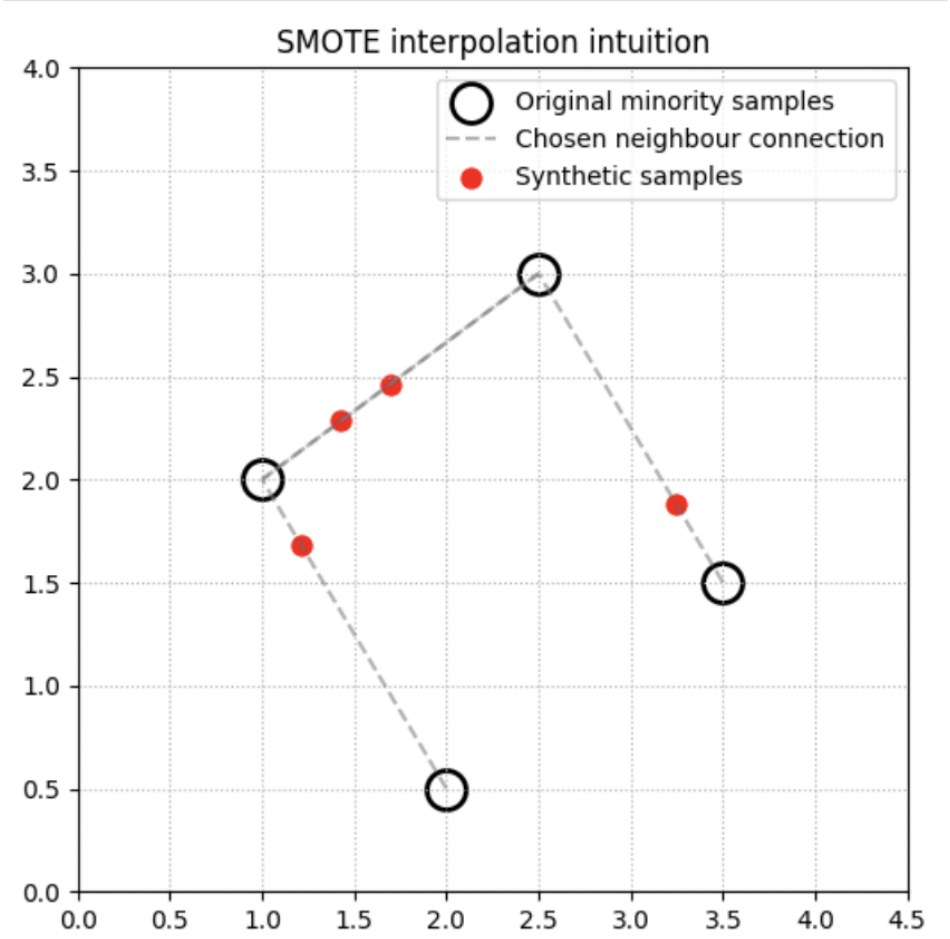
[1] SMOTE: synthetic minority over-sampling technique NV Chawla, KW Bowyer, LO Hall, WP Kegelmeyer. Journal of artificial intelligence research, 2002

# **Oversampling:** SMOTE (intuition)

**1. Choose a minority instance**
Randomly pick a sample from the minority class.

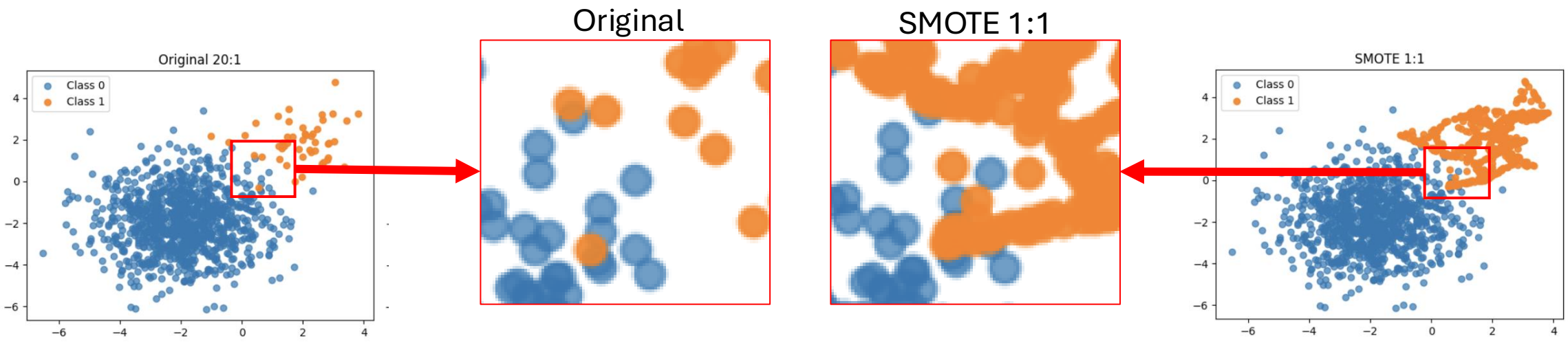
**2. Find a nearby minority neighbour**
Identify one of its *k nearest neighbours* from the minority class.

**3. Create a synthetic point between them**
Generate a new instance somewhere along the line segment connecting the two points.




[1] SMOTE: synthetic minority over-sampling technique NV Chawla, KW Bowyer, LO Hall, WP Kegelmeyer. Journal of artificial intelligence research, 2002

# **Oversampling:** SMOTE limitations




- Creates synthetic instances that are not real, just interpolations between existing minority samples
- That becomes a problem when those synthetic points are **not plausible** or **do not reflect the true data distribution**

# Cost-sensitive (Class weighting)

- Assign different importance (weights) to classes during training
- Misclassifying minority class instances incurs a higher penalty
- Encourages the model to pay more attention to rare classes

**Example: (Balanced) Decision trees**

- Class weights modify how split quality is evaluated during tree construction
- Minority class instances have greater influence on the splits chosen during tree construction

# Cost-sensitive: *(Balanced)* Decision Trees*

1. **Start with class weights**
   Example: majority class weight = 1, minority class weight = 20.
2. **For each candidate split, count weighted class frequencies**
   Instead of counting each sample as 1, count it according to its class weight.
3. **Compute impurity using weighted counts**
   Use the weighted class proportions to compute entropy or Gini.
4. **Compare candidate splits as usual**
   The tree still chooses the split with the best impurity reduction.

*** Inside a Random Forest or any other ensemble:** repeat this for every tree.

# Cost-sensitive: *(Balanced)* Decision Trees*

1. **Start with class weights**
   Example: majority class weight = 1, minority class weight = 20.
2. **For each candidate split, count weighted class frequencies**
   Instead of counting each sample as 1, count it according to its class weight.
3. **Compute impurity using weighted counts**
   Use the weighted class proportions to compute entropy or Gini.
4. **Compare candidate splits as usual**
   The tree still chooses the split with the best impurity reduction.

Class weighting **does not change the labels or create new data**. It changes how much each example counts when evaluating splits.

*** Inside a Random Forest or any other ensemble:** repeat this for every tree.

# Wrap-up

- Feature selection
  - Filter, wrappers and embedding
  - Not so popular anymore, still useful if the goal is interpretability (and feature set is not massive – prohibitive for wrapper methods)
- Evaluation
  - K-fold CV: more confidence in the performance estimation on unseen data

**Next lecture:** Dimensionality reduction & Imbalanced classification